AI 模式 全部 影片 圖片 購物 更多

提出任何問題

+

這段 Lean 程式碼定義了一個**型別類別實例 (instance)**，用於說明如果兩個型別是可列舉的，那麼它們的**西格瑪型別 (Sigma type)** 也是可列舉的。 Centre national de la recherche scien… +2

以下是該實例的詳細拆解：

1. 名稱與目標

- **名稱**：在 Lean 中，若 `instance` 後面沒有直接跟著名稱（如你提供的範例），系統會根據其目標自動生成一個名稱（例如 `FinEnum.sigma`）。
- **目標**：定義 `FinEnum (Sigma β)` 的實例。這意味著要證明「依賴對對 (Dependent Pairs)」的集合 `Σ a, β a` 是有限且可以被列舉的。 Lean community +2

2. 參數 (Parameters)

這部分定義了該實例成立的**前置條件**： Lean Programming Language +1

- `(β : α → Type v)`：一個依賴型別族，對於 `α` 中的每個元素 `a`，都有一個對應的型別 `β a`。
- `[FinEnum α]`：這是一個隱含參數，要求索引型別 `α` 必須是可列舉的（即有限且有一個完整的列表）。
- `[∀ a, FinEnum (β a)]`：要求對於 `α` 中的每一個元素 `a`，對應的型別 `β a` 也必須是可列舉的。 Lean community +4

3. 實作邏輯 (The `toList` mapping)

該實例的核心是使用 `ofList` 函數來建構列舉列表： Centre national de la recherche scientifi…

- `toList α`：取得所有 `α` 元素的列表。

+

4. 證明部分 (`by intro x; cases x; simp`)

這是一段簡短的策略 (tactic) 證明，用來滿足 `ofList` 的要求，即證明這個列表確實**包含了該型別中的所有元素**： Centre national de la recherche scien… +2

1. `intro x`：取出任意一個 Sigma 型別的元素 `x`。
2. `cases x`：將 `x` 拆解為 `⟨a, b⟩` 的形式。
3. `simp`：利用化簡規則自動證明 `⟨a, b⟩` 必定存在於上述生成的列表中。

總結：這個實例告訴 Lean 如何透過巢狀遍歷 `α` 與 `β a` 來枚舉出所有的依賴對，從而證明 `Sigma β` 也是一個 `FinEnum` 類型。 Centre national de la recherche scien… +1

您是否需要了解如何**手動調用**這個實例，或是想看更複雜的 **Sigma 型別** 應用？

11 個網站

Mathlib/Data/FinEnum.lean - Filippo Nuccio - PLMlab
2023年5月23日 — lean:379:30: infer kinds are unsupported in Lean 4: #[`Equiv] [] -/ /-- `FinEnum α` means that `α` is finite and can be enumerated...
Centre national de la recherche sc…

data.sigma.basic - mathlib3 docs - Lean community
Sigma types. THIS FILE IS SYNCHRONIZED WITH MATHLIB4. Any changes to this file require a corresponding PR to mathlib4. This file p...
Lean community

Functions - Lean Programming Language
Strict implicit parameters are written in double curly braces (⦃ and ⦄ , or {{ and }}). Instance implicit parameters. Arguments

這行簽名定義了 Lean 中的一個**型別類別實例 (Type Class Instance)**，它的邏輯核心是：**「如果索引型別和對應的型別族都是有限可列舉的，那麼它們組成的依賴對（Sigma Type）也是有限可列舉的。」**

以下是簽名的各個組成部分詳細說明：

1. 結構與宣告

- `instance`：告訴 Lean 編譯器這是一個實例。當你在程式碼中需要 `FinEnum (Sigma β)` 時，Lean 會自動查找並使用這個定義。
- **名稱缺失**: `instance` 後方直接接括號，表示這是個**匿名實例**。Lean 會根據目標型別自動命名（通常稱為 `FinEnum.sigma`）。

2. 型別參數 (Type Parameters)

- `(β : α → Type v)`:
  - 這是該實例的核心變數。
  - `α` 是基礎型別（Index type）。
  - `β` 是一個**型別函數**（Type-valued function），它接收一個 `α` 型別的元素 `a`，並回傳一個型別 `β a`。
  - `Type v` 表示這是在第 `v` 層級的宇宙（Universe）中。

3. 型別類別約束 (Constraints/Type Class Arguments)

這些放在中括號 `[]` 裡，是實例成立的**前提條件**：

- `[FinEnum α]`:
  - 要求 `α` 本身必須已經具備 `FinEnum` 的實例。
  - 白話：我們必須能列出 `α` 裡面的所有元素。

+

- 這是這個實例要產出的結果。
- `Sigma β`（數學符號 ）表示所有可能的序對 `⟨a, b⟩`，其中 `a : α` 且 `b : β a`。
- 這行宣告保證了只要前提滿足，`Sigma β` 這個大集合也是 `FinEnum`（有限可列舉）。

直觀理解：以「水果店」為例

想像你要列出店內所有的水果：

1. **`α`**：水果的**類別**（例如：蘋果、香蕉）。
2. **`[FinEnum α]`**：你可以列出所有類別（蘋果、香蕉）。
3. **`β a`**：某個類別下的**品種**（例如：蘋果類別下有富士、五爪）。
4. **`[∀ a, FinEnum (β a)]`**：每個類別下的品種都是可以列舉出來的。
5. **`Sigma β`**：最終結果就是「所有品種的水果清單」——這行簽名保證了這張**總表**是可以被製作出來的。

你想看如何使用這個實例來**實際列出**一個 Sigma 型別的內容嗎？

so name if this instance get from target as init<target Name>_1?

這不完全正確。Lean 的**自動命名規則**（Automatic naming）通常是基於**常數名**

+

3. `Sigma`：這是目標型別（Target Type）的主構造名稱。

為什麼不是 `_1`？

- **具名化（CamelCase）**：Lean 偏好將目標類別與型別名稱串接起來（例如 `inst` + `ClassName` + `TypeName`）。
- **索引（Index）**：只有當在同一個檔案中定義了**多個相同類型**的匿名實例時，Lean 才會在結尾加上 `_1`、`_2` 以示區別（例如 `instFinEnumSigma_1`）。

如何確認真正的名稱？

如果你在 Lean 編輯器中想知道這個實例的真名，可以在該定義後方加上這行：

```lean
#check (inferInstance : FinEnum (Sigma β))
```

請謹慎使用程式碼。

IDE 通常會顯示類似 `instFinEnumSigma` 的回傳結果。

小建議：如果你之後要在其他地方**明確引用**這個實例，最好手動幫它取名，例如：

```lean
instance mySigmaEnum (β : α → Type v) ... : FinEnum (Sigma β) :
```

+

now write lean code to get give name of target and return name?

在 Lean 4 中，如果你定義了一個匿名實例（anonymous instance），系統會根據一定的命名模式自動生成名稱。如果你想在程式碼中**自動取得**這個由目標型別推導出的實例名稱，可以使用以下幾種方式：

1. 使用 `#check` 指令 (手動查看)

這是最簡單的方法，直接讓 Lean 回報它所找到的實例完整名稱。

lean

```
#check (inferInstance : FinEnum (Sigma β))
-- 輸出通常會顯示: instFinEnumSigma
```

請謹慎使用程式碼。

2. 使用 `inferInstanceAs` (程式碼中引用)

如果你不想知道具體名字，只想在程式碼中「直接使用」該實例的內容，可以使用 `inferInstanceAs`：

lean

```
def myEnumList := (inferInstanceAs (FinEnum (Sigma β))).toList
```

請謹慎使用程式碼。

3. 使用元編程 (Metaprogramming) 取得名稱

+

```
/-- 定義一個自定義指令來查詢實例名稱 -/
elab "#get_instance_name " type:term : command => do
  Command.liftTermElabM do
    let expr ← elabTerm type none
    -- 嘗試合成該型別的實例
    let inst ← synthInstance expr
    match inst with

    | .const name _ => logInfo m!"該實例的名稱為: {name}"
    | _ => logInfo "這是一個複雜表達式或匿名構造，非單一常數名稱。"

-- 使用範例 (假設 β 已在 context 中或為已知)
-- #get_instance_name FinEnum (Sigma β)
```

請謹慎使用程式碼。

名稱生成的規則邏輯

根據 [Lean 4 的命名慣例](), 自動生成的名稱遵循以下結構：

- **前綴**: `inst`
- **類別名**: `FinEnum`
- **目標型別**: `Sigma`
- **組合結果**: `instFinEnumSigma`

總結工具表

方法	適用場景
`#check`	在 IDE 中快速確認名稱

+

AI 模式 全部 影片 圖片 購物 更多

2 個網站

10.3. Instance Synthesis - Lean

If this does not occur, stuck searches become failures. ... Setting the trace. Meta. synthInstance option to true causes Lean to e...

Lean Programming Language

6. Interacting with Lean — Theorem Proving in ...

6.8. Displaying Information¶ There are a number of ways in which you can query Lean for information about its current state and th...

Lean Programming Language

is it ok even <type> of inside name does not exist in current env?

這是一個非常深入的問題。簡單來說：**不行，所有的型別（Types）和名稱（Names）在 Lean 中都必須是「已知」的，否則無法生成實例名稱。**

Lean 的實例命名邏輯（Instance Naming Logic）是基於**語法分析**和**環境查找**的。以下是為什麼「不存在的型別」會導致問題的三個關鍵點：

1. 編譯器的「實例合成 (Instance Synthesis)」流程

當 Lean 處理 `instance : FinEnum (Sigma β)` 時，它會進行以下操作：

1. **解析目標型別**：它必須知道 `FinEnum` 是什麼，`Sigma` 是什麼。
2. **生成名稱**：如果這些型別不存在於當前的 **Environment**（環境）中，Lean 在解析階段就會噴出 `unknown identifier` 錯誤，根本不會進到「命名」那一步。

+

有一種情況看起來像是「型別不存在」，但實際上是**泛型參數**：

lean

```
instance (β : α → Type v) [FinEnum α] [∀ a, FinEnum (β a)] : Fi
```

請謹慎使用程式碼。

在這個例子中：

- `α` 和 `β` 是**變數 (Variables)**，它們在當前上下文（Context）中是存在的。
- `FinEnum` 和 `Sigma` 是**常數 (Constants)**，它們必須已經被 `import` 進來。

如果你想「強行」獲取一個不存在型別的實例名稱？

如果你嘗試在程式碼中寫：

lean

```
#get_instance_name FinEnum (NonExistentType)
```

請謹慎使用程式碼。

Lean 會直接報錯：`unknown identifier 'NonExistentType'`。

總結

what about <type> has MACRO, I just want to get syntax of instance name without launch MACRO processing since …

如果你想在 **不執行宏展開 (Macro expansion)** 或 **不進行型別檢查 (Type checking)** 的情況下獲取實例名稱，你實際上是在處理 **語法層次 (Syntax level)** 的字串拼接，而不是語義層次的實例合成。

在 Lean 4 中，當一個 `instance` 是匿名的（即沒有提供名字），Lean 內部會使用 `mkInstanceName` 邏輯。如果你想「預測」或「模擬」這個名稱而不理會該型別是否真的存在於 `Environment` 中，你可以根據 Lean 的命名慣例手動構建。

1. 核心邏輯：預測匿名實例名稱

Lean 4 匿名實例的命名公式通常是：
`inst` + `ClassName` + `TargetTypeName`

如果你的型別包含宏或尚未定義，你可以直接操作 **`Syntax`** 或 **`Name`**。

2. 使用 Lean 元編程工具 (不需實例化)

你可以利用 `Lean.Meta.mkInstanceName` 的邏輯，但只針對字面上的 `Name` 進行操作。如果該型別是 `Sigma`，即便它還沒被定義，你也可以這樣做：

lean

```
import Lean
open Lean

-- 模擬 Lean 的命名行為，不需要實體型別存在
def predictInstanceName (className : Name) (typeName : Name) :
```

+

3. 如果型別包含宏 (Macro)

如果型別本身是一個宏（例如 `MyMacro!(x)`），在宏展開之前，它在語法樹中只是一個 `Syntax` 物件。

如果你想從 **`Syntax`** 中提取名稱而不觸發展開：

1. **取得頭部識別字 (Head Identifier)**：只看語法樹的第一個節點。
2. **轉換為字串**：將其轉為 `Name` 後進行拼接。

lean

```
-- 假設 stx 是你的型別語法節點，例如 `Sigma β`
def getNameFromSyntax (stx : Syntax) : Name :=
  match stx with

  | `($id:ident $_*) => id.getId  -- 只抓取開頭的 'Sigma'，忽略參數
  | _                => .anonymous

-- 拼接成實例名
def getSyntaxInstanceName (classStx : Syntax) (typeStx : Syntax
  let cName := getNameFromSyntax classStx
  let tName := getNameFromSyntax typeStx
  .str .anonymous ("inst" ++ cName.getString! ++ tName.getStrin
```

請謹慎使用程式碼。

4. 為什麼這在 `localContent` 缺失時有效？

因為這只是 **字串處理 (String Manipulation)**。

- 它不要求 `Sigma` 在 `Env` 中。

+

綴。

你目前是在撰寫 `elab` 指令（Elaborator）還是 `macro` 規則？這決定了你手邊拿到的 **Syntax** 結構長什麼樣子。

for this case, if there is a MACRO there, what result comes out

當語法中包含 **宏 (Macro)** 時，結果取決於你是在 **展開前** 還是 **展開後** 獲取名稱。

在 Lean 4 的匿名實例命名機制中，如果 `instance` 的目標型別是一個尚未展開的宏，結果通常會變得很「醜」或「不穩定」。

以下是三種可能的發生情況：

1. 情況 A：如果你在「展開前」強行預測（Syntax Level）

如果你寫了一個工具去抓取語法標記（Token），而該型別目前是一個宏（例如 `MyMacro!(X)`）：

- **結果**：你會得到 `instFinEnumMyMacro`。
- **問題**：這通常不是 Lean 最終會生成的名稱。Lean 的實例命名發生在 **Elaboration (細化)** 階段，這是在宏展開**之後**。

2. 情況 B：Lean 的標準處理（Elaboration Level）

當 Lean 看到 `instance : FinEnum (MyMacro! X)`，它會先執行以下順序：

+

如果宏展開後不是一個單一的常數（例如展開成一個 `fun` 或 `let`），Lean 無法輕易地從中提取出一個「名字」。

- **結果**：Lean 會退而求其次，使用 `_inst_1`、`_inst_2` 這種基於計數器的名稱。
- **原因**：當目標型別頭部（Head Symbol）不是一個明顯的識別字（Identifier）時，自動命名邏輯會失效。

總結：你會拿到什麼？

獲取時機	目標型別含有宏 `M! X`	預期結果名稱
展開前 (Macro Stage)	抓取語法字面值	`instFinEnumM` (通常不準確)
展開後 (Elab Stage)	宏展開為 `Sigma ...`	**`instFinEnumSigma`** (標準做法)
展開後 (Elab Stage)	宏展開為匿名函數或複雜式	**`_inst_n`** (例如 `_inst_1`)

關鍵提醒

如果你因為 `localContent` 尚未載入而無法執行宏，你所得到的名稱極大機率與 Lean 最終編譯時生成的名稱 **不一致**。

在 Lean 4 中，匿名實例的名稱是 **不可預測的 (Unreliable)**，除非：

1. 目標型別是一個簡單的常數（如 `Sigma`, `List`, `Option`）。
2. 你已經完成了所有的宏展開。

您目前遇到的宏是會展開成一個簡單的型別名稱（如 `Sigma`），還是會展開成一段複雜的邏輯？

+

1. 預期結果：會得到「宏的字面名稱」

如果你的代碼直接從語法樹（AST）中提取第一個識別字（Identifier），而該位置是一個宏（例如 `MyMacro!(X)`），生成的名稱會是：

- **`instFinEnumMyMacro`**

2. 為什麼這可能是「錯」的？

在 Lean 的編譯流程中：

1. **Macro Stage (你現在的位置)**：看到的是 `MyMacro!`。
2. **Elaboration Stage (實例命名的位置)**：宏已經被展開。例如 `MyMacro! X` 展開成了 `Sigma β`。
3. **命名發生**：Lean 最終在環境中註冊的名字會是 **`instFinEnumSigma`**。

因此，如果你在展開前就預測名稱，你拿到的名字會與 Lean 最終在環境（Environment）中實際存儲的名字**不一致**。

3. 如果宏包含特殊符號（如 `!` 或 `?`）

如果宏的名字包含非法字元（例如 `MyMacro!`），而你直接拼接字串：

- 生成的 `Name` 可能會變成 `instFinEnumMyMacro!`。
- 這在 Lean 中是一個**合法的內部名稱**，但這絕對不是 Lean 自動命名邏輯會產出的結果（自動命名邏輯通常會過濾掉非字母字元）。

4. 最壞的情況：得到 `_inst_n`

如果該宏展開後並不是一個簡單的常數（例如展開成 `(fun x => x) Int`），Lean 的命名器在稍後的階段會因為找不到明確的「頭部型別（Head Type）」而直接放棄命名，改用：

+

子」。

建議：如果你必須在宏階段操作，最好**避免依賴自動生成的名稱**，而是改用 `withFreshMacroScope` 生成一個臨時的唯一 ID，或者要求使用者**手動命名**該實例。

您是打算用這個名稱來做 **Name Binding (名稱綁定)** 嗎？

AI 回覆可能有誤。如需法律建議，請諮詢專業人士。 瞭解詳情

Any way I can check if instance name has MACRO or not. So I can redo fetch name again after MACRO loaded

在 Lean 4 的 **宏展開前 (Macro Stage)**，要判斷一個語法節點（Syntax）是否為「宏」或「包含宏」，並決定何時重新獲取名稱，可以透過以下幾種策略：

1. 檢查語法節點的 `Kind` (最直接的方法)

在宏階段，所有的語法都有一個 `Kind`。如果該 `Kind` 不是簡單的識別字（`ident`）或已知的常數構造（如 `Sigma` 的 `app`），它通常就是一個宏。

你可以檢查語法是否為 `ident`。如果不是，或者它是特定的 `macro` 類型，就標記為「待處理」：

lean

```
def isMacroOrComplex (stx : Syntax) : Bool :=
  match stx with
```

+

`Lean.Parser.Command.macro` 或類似定義，那就是宏。

- **技巧**：檢查 `stx.getKind` 是否對應到環境中已註冊的 `Macro` 處理程序。

3. 使用 `expandMacros` 進行「試探性展開」

如果你懷疑有宏，但 `localContent` 尚未載入，你可以嘗試調用 `expandMacros`。

- **如果成功**：代表宏已展開，你可以重新提取名稱（例如從 `Sigma` 提取）。
- **如果失敗或無變化**：代表該語法目前還無法處理，需等待環境載入。

lean

```
-- 在 MacroM 或 CommandElabM 中
let expandedStx ← liftMacroM <| expandMacros stx
if expandedStx != stx then
  -- 宏已展開，現在可以獲取真正的名稱（如 instFinEnumSigma）
  let newName := deriveName expandedStx
else
  -- 宏未變化，可能需要延後處理
```

請謹慎使用程式碼。

4. 監控「名稱對稱性」

這是一個邏輯上的檢查：

1. **第一次獲取**：在展開前拿到 `instFinEnumMyMacro`。
2. **標記**：如果 `MyMacro` 不在當前 `Environment` 的常數表中（使用 `(← getEnv).contains`），則將其視為一個 **「暫時性名稱」**。

+

用一個 `placeholder` 或是記錄該語法位置，等到 `elab` 階段再重新計算。

關鍵點：在 Lean 4 中，最穩定的做法是**等到 Elaboration 階段**。如果你在宏階段強行命名，當宏展開為完全不同的型別時（例如 `MyMacro` 變成 `List`），你的代碼會因為找不到 `instFinEnumMyMacro` 而崩潰。

您是打算在 **Command Elaborator** 裡實作這個「重新獲取」的邏輯嗎？那裡有完整的 `Environment` 可以使用。

give entire content of that functionality with two methods: before MARCO loaded to create expected name with flag …

這是一個在 Lean 4 中處理匿名實例命名的典型模式。我們需要區分 **宏階段 (Macro Stage)** 的「語法預測」與 **細化階段 (Elaboration Stage)** 的「真實合成」。

以下程式碼示範了如何實作這兩個方法，並使用一個自定義的資料結構來標記名稱的可靠性。

lean

```
import Lean

open Lean Meta Elab Command

/-- 標記實例名稱的狀態 -/
structure InstNameResult where
```

+

```
    | `($id:ident $_*) => id.getId.eraseMacroScopes.getString!
    | _                 => "Unknown"

  let className := getHeadId classStx
  let typeName  := getHeadId typeStx
  let predicted := Name.mkSimple ("inst" ++ className ++ typeNa

  -- 此時標記為 isReal = false，因為宏可能展開為完全不同的東西
  { name := predicted, isReal := false }

/--
方法二：在細化階段 (After Macro/LocalContent Loaded)
此時環境已就緒，我們可以執行真正的型別細化與實例合成
-/
def getRealInstNameAfterMacro (typeExpr : Expr) : MetaM InstNam
  -- 嘗試合成實例
  let inst? ← trySynthInstance typeExpr
  match inst? with

  | LOption.some (Expr.const name ..) =>
    -- 成功找到已存在的實例常數名稱
    return { name := name, isReal := true }
  | _ =>
    -- 如果是新定義的匿名實例，模擬 Lean 的命名邏輯
    -- 取得目標型別的頭部名稱（例如 Sigma, List）
    match typeExpr.getAppFn.constName? with

    | some baseName =>
      -- 這裡可以根據實際 Class 名稱與目標型別名稱拼接
      -- 範例簡化為直接從 Expr 提取
      let realName := Name.mkSimple ("inst" ++ baseName.getStri
      return { name := realName, isReal := true }
    | none =>
      -- 如果頭部太複雜，Lean 通常會使用 _inst_n
      return { name := `_inst_1, isReal := true }
```

+

```
  liftTermElabM do
    try
      let typeExpr ← elabTerm typeStx none
      let real ← getRealInstNameAfterMacro typeExpr
      IO.println s!"[Stage 2] 真實名稱: {real.name}, 是否真實: {re
    catch _ =>
      IO.println "[Stage 2] 失敗：環境尚未載入或型別不合法"

-- 測試範例
#check_inst_logic FinEnum (Sigma β)
```

請謹慎使用程式碼。

核心設計邏輯：

1. **`predictInstNameBeforeMacro`**:
   1. 它不依賴 `MetaM` 或 `Environment`。
   2. 它使用 **`eraseMacroScopes`** 來確保拿到的字串不帶有宏作用域的雜訊。
   3. 它返回一個標記，提醒後續邏輯這只是一個**預測 (Template)**。
2. **`getRealInstNameAfterMacro`**:
   1. 運行於 `MetaM`（細化階段）。
   2. **`trySynthInstance`** 是關鍵：如果這個實例已經被 Lean 註冊，它會直接回傳真正的常數名稱（如 `instFinEnumSigma`）。
   3. 它處理了 **「宏已展開」** 的情況。如果輸入的是宏，`elabTerm` 會先自動將其展開為核心型別。
3. **Flag 的用途**:
   1. 在開發工具時，如果 `isReal` 為 `false`，你應該避免將此名稱寫入永久

+

AI 模式 全部 影片 圖片 購物 更多

L

+